NUTRIGUIDE AI

A MINI-PROJECT REPORT

Submitted by

PREETHI KN **240701398**

RAGAVI N **240701407**

in partial fulfillment of the award of the degree

of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING



RAJALAKSHMI ENGINEERING COLLEGE, CHENNAI

An Autonomous Institute

CHENNAI

NOVEMBER 2026

BONAFIDE CERTIFICATE

Certified that this project **"NUTRIGUIDE AI"** is the bonafide work of **" PREETHI KN, RAGAVI N"** who carried out the project work under my supervision. We express our sincere thanks to our beloved and honorable chairman **MR. S. MEGANATHAN** and the chairperson **DR. M.THANGAM MEGANATHAN** for their timely support and encouragement. We are greatly indebted to our respected and honorable principal **Dr. S.N. MURUGESAN** for his able support and guidance. No words of gratitude will suffice for the unquestioning support extended to us by our Head Of The Department **Dr. J. MANORANJINI** for being ever supporting force during our project work We also extend our sincere and hearty thanks to our internal guide **MS. DEEPA,** for her valuable guidance and motivation during the completion of this project. Our sincere thanks to our family members, friends and other staff members of computer science engineering.

SIGNATURE

MS DEEPA

ASSISTANT PROFESSOR SG

Dept. of Computer Science and Engineering,
Rajalakshmi Engineering College
Chennai

This mini project report is submitted for the viva voce examination to be held on _____

INTERNAL EXAMINER **EXTERNAL EXAMINER**

Abstract

NUTRIGUIDE AI

In today's fast-paced world, nutrition and fitness have become significant challenges for individuals across all age groups and lifestyles. Despite the availability of numerous diet and fitness applications, most offer static, one-size-fits-all meal plans and generic workout routines that fail to account for a user's specific medical conditions, allergies, or dietary restrictions. Recognizing this gap, our team has developed NutriGuide AI, a database-driven, AI-powered application designed to support individuals in managing their nutrition and fitness effectively through genuinely personalized, verifiable recommendations.

The primary objective of this project is to provide users with tailored meal and workout plans based on their specific health profiles, goals, and constraints. The system maintains detailed records of user profiles, dietary restrictions, meal and food data, workout history, and progress logs. By organizing this data efficiently and reasoning over it using classical Artificial Intelligence techniques — including search algorithms, constraint satisfaction, logical inference, and machine learning — the platform enables users to access personalized meal plans, guided workout routines, food identification through scanning, and an explainable AI chatbot with ease.

Unlike conventional recommendation systems that rely purely on static lookups or opaque generative outputs, NutriGuide AI guarantees that every recommendation is traceable to explicit rules and constraints, ensuring that a suggested meal never violates a user's allergies or medical limitations. This structured, explainable approach empowers users and, potentially, nutritionists and fitness practitioners, to deliver targeted support, monitor engagement, and refine recommendations over time based on real feedback.

By offering a structured, transparent, and accessible approach to nutrition and fitness planning, the platform not only enhances individual well-being but also demonstrates how classical AI reasoning can be combined with modern generative AI to build trustworthy, health-sensitive applications. Through this initiative, we aim to bridge the gap between generic wellness apps and genuinely intelligent, personalized health guidance, fostering a culture of informed, proactive self-care.

ACKNOWLEDGEMENT

TABLE OF CONTENT

FIGURE NO.	TITLE	PAGE NO.
1	INTRODUCTION	5
2	SYSTEM SPECIFICATION	7
3	MODULE DESCRIPTION	8
4	SAMPLE CODES	10
5	PAGES SCREENSHOTS	13
6	CONCLUSION AND FUTURE ENHANCEMENT	16

CHAPTER 1

1.1 INTRODUCTION

NutriGuide AI is a personalized nutrition and fitness recommendation system designed to help individuals plan meals and workouts that respect their health conditions, dietary restrictions, and fitness goals. The platform provides users with a personalized dashboard that displays daily meal plans, workout routines, food-scan results, and progress analytics. Rather than offering generic, one-size-fits-all diet charts, the system applies a set of classical Artificial Intelligence techniques — search, constraint satisfaction, logical inference, game-theoretic search, and machine learning — to reason about each user's profile and produce recommendations that are explainable and verifiably consistent with stated constraints such as allergies, medical conditions, and calorie targets.

This structured approach empowers users to take proactive steps toward healthier eating and consistent exercise, while giving the project itself a genuine, demonstrable AI core rather than a set of static lookup tables

1.2 SCOPE OF THE WORK

NutriGuide AI is designed to help individuals access personalized diet and workout support amid the growing demand for health-conscious digital tools. It offers quick, structured access to meal planning, workout scheduling, food identification, conversational guidance, and progress tracking, all tailored to the user's medical and lifestyle profile. By combining rule-based reasoning with search and optimization, the system helps users manage nutrition and fitness more effectively without requiring a dietitian for every decision. The scope covers five core areas mapped directly to classical AI foundations: intelligent agents and search, constraint satisfaction and game-based reasoning, logical inference, and machine learning applications

1.3 PROBLEM STATEMENT

Most consumer diet and fitness apps either present static, pre-written meal plans or rely entirely on third-party generative AI calls with no verifiable reasoning behind a recommendation. Users with medical conditions such as diabetes, pregnancy, or allergies need guarantees — not just suggestions — that a recommended meal actually satisfies their constraints. This project addresses that gap by building a recommendation engine whose decisions are produced by explicit, inspectable

algorithms (constraint satisfaction, search, and rule-based inference), so that every recommendation can be traced back to the constraints and rules that produced it.

1.4 AIM AND OBJECTIVES OF THE PROJECT

The main objective of this project is to provide personalized, constraint-safe meal and workout recommendations based on an individual user's health profile. The system maintains structured records of user profiles, meal and food data, workout splits, and interaction logs, and uses this data to drive multiple AI subsystems. Specific objectives include:

- To design a goal-based intelligent agent architecture that perceives user profile data and queries, and dispatches to the appropriate recommendation subsystem.
- To implement uninformed and informed search (BFS, Uniform-Cost, A*) to construct daily meal plans that meet calorie and macro targets.
- To implement a Constraint Satisfaction Problem (CSP) solver with backtracking and constraint propagation to guarantee that recommended meals never violate allergy or medical constraints.
- To implement local search (simulated annealing) for weekly meal-plan optimization.
- To implement a rule-based expert system using forward and backward chaining for dietary reasoning and an explainable chatbot.
- To implement Minimax, Alpha–Beta pruning, and Monte Carlo Tree Search as a self-contained AI Lab module demonstrating game-playing search.
- To implement supervised learning (regression), unsupervised learning (k-means clustering), and a lightweight reinforcement-learning feedback loop for progress prediction and personalization.
- To integrate a Generative AI layer (Gemini API) for natural-language explanations, proxied securely through the backend.

CHAPTER 2

SYSTEM SPECIFICATIONS

2.1 HARDWARE SPECIFICATIONS

Processor	:	RMD RYZEN
Memory Size	:	8 GB DDR4 (Minimum)
HDD	:	1 TB HDD(Minimum)

2.2 SOFTWARE SPECIFICATIONS

Operating System	:	WINDOWS 11
Front – End	:	React.js
Back - End	:	MongoDB
Language	:	Python
API	:	FastAPI
Authentication	:	JWT Authentication

CHAPTER 3

MODULE DESCRIPTION

The application consists of two roles — a registered User and, optionally, an Admin who manages the meal, food, and workout reference data stored in MongoDB. Once logged in, a user's profile drives every downstream recommendation. The system is organized into the following functional modules, each backed by a specific classical AI technique from the syllabus:

3.1 AUTHENTICATION AND PROFILE MODULE

Handles registration, login (JWT-based), and the user's health profile (allergies, medical condition, calorie/protein targets, glycemic limit). This profile is the shared percept source for every other module.

3.2 INTELLIGENT AND AGENT LAYER

A goal-based agent wrapper (perceive → decide → act) that merges profile data with each new query or event and dispatches to the correct subsystem below. Implements Unit I — Intelligent Agents.

3.3 MEAL PLANNER MODULE (SEARCH + ASP)

Generates a day's meal plan using Breadth-First Search / Uniform-Cost Search for a first valid plan, A* Search with a calorie-gap heuristic for an optimized plan, and a backtracking Constraint Satisfaction Problem solver with constraint propagation (domain pruning for allergies and glycemic-index limits) to guarantee hard constraints are never violated. A weekly optimizer uses simulated annealing (local search) to minimise deviation from weekly calorie targets. Implements Units I, II and III.

3.4 FOOD SCANNER MODULE

Identifies a scanned food item and returns a ranked, uncertain candidate list with confidence scores. Models the problem as search in a partially observable

environment: if top confidence is below threshold, the agent requests user confirmation instead of committing. Implements Unit II.

3.5 AI CHATBOT MODULE

A rule-based expert system over propositional facts derived from the user's profile and query (e.g. hasCondition(diabetes), gi_high(food)). Forward chaining derives all applicable recommendations/rejections; backward chaining reconstructs the reasoning trail behind any single recommendation for a 'Why this recommendation' explanation panel. Implements Unit IV.

3.6 AI LAB MODULE(GAME)

A self-contained demonstration page implementing Minimax, Alpha–Beta pruning, and Monte Carlo Tree Search on a small two-player game board, independent of the nutrition domain. Implements Unit III

3.7 ANALYTICS AND MACHINE LEARNING MODULE

Applies linear regression over logged weight/calorie history to project future trends, k-means clustering over user macro profiles for 'users like you' recommendations, and a lightweight reinforcement-learning weight update over accepted/rejected meal suggestions to personalise future rankings. Implements Unit V.

CHAPTER 4

SAMPLE CODING

4.1 Constraint Satisfaction — Backtracking with Constraint Propagation

```
def propagate_domains(slots, candidates, profile):
    allergies = set(a.lower() for a in profile.get("allergies", []))
    pruned = {}
    for slot in slots:
        pruned[slot] = [
            m for m in candidates[slot]
            if not allergies.intersection(t.lower() for t in m.get("tags", []))
            and not (profile.get("health_condition") == "diabetes"
                    and m["glycemic_index"] > profile.get("glycemic_target_max", 55))
        ]
    return pruned


def backtrack(slots, domains, assignment, profile):
    if len(assignment) == len(slots):
        total_cal = sum(m["calories"] for m in assignment.values())
        target = profile.get("daily_calories_target", 2000)
        if abs(total_cal - target) <= target * 0.1:
            return dict(assignment)
        return None
    slot = slots[len(assignment)]
    for meal in domains[slot]:
        assignment[slot] = meal
        result = backtrack(slots, domains, assignment, profile)
        if result:
```

```
            return result
        del assignment[slot]
    return None
```

4.2 Informed Search — A* Meal Plan Search

```
def build_day_plan_astar(slots, candidates, calorie_target):
    frontier = [(0, 0, [], 0)]   # f_score, tie-breaker, chosen, total
    while frontier:
        f, _, chosen, total = heapq.heappop(frontier)
        if len(chosen) == len(slots):
            return chosen
        slot = slots[len(chosen)]
        for meal in candidates.get(slot, []):
            new_total = total + meal["calories"]
            g = abs(calorie_target - new_total)
            h = abs(calorie_target - new_total) / max(len(slots)-len(chosen)-1, 1)
            heapq.heappush(frontier, (g + h, len(frontier), chosen + [meal], new_total))
```

4.3 Logical Reasoning — Forward Chaining

```
def forward_chain(facts, rules):
    derived = set(facts)
    changed = True
    while changed:
        changed = False
        for conditions, conclusion in rules:
            if all(c in derived for c in conditions) and conclusion not in derived:
                derived.add(conclusion)
                changed = True
    return derived
```

4.3 Logical Reasoning — Forward Chaining

```
def forward_chain(facts, rules):
    derived = set(facts)
    changed = True
    while changed:
        changed = False
        for conditions, conclusion in rules:
            if all(c in derived for c in conditions) and conclusion not in derived:
                derived.add(conclusion)
                changed = True
    return derived
```

CHAPTER 5

SCREEN SHOTS

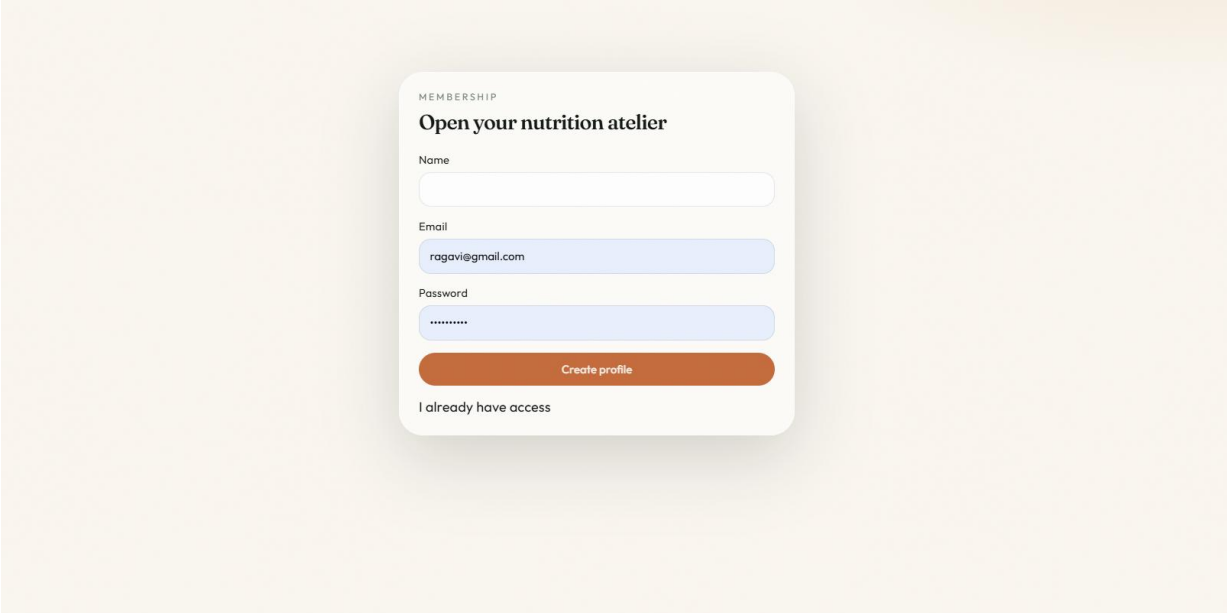


Fig 5.1 Login / Signup Page

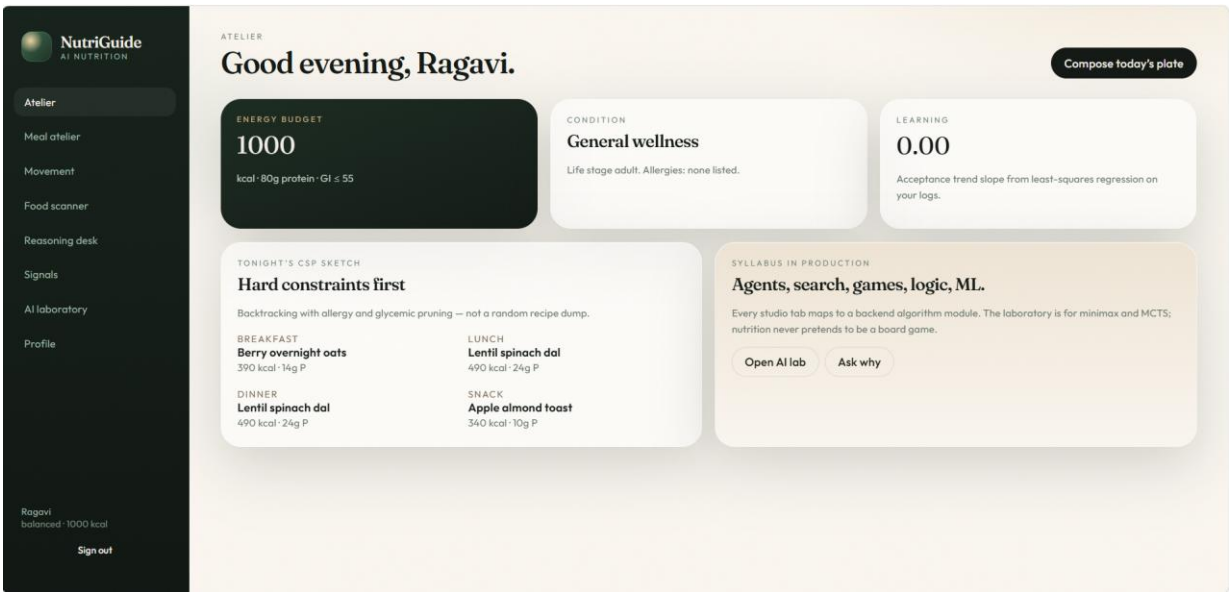


Fig 5.2 Dashboard Overview

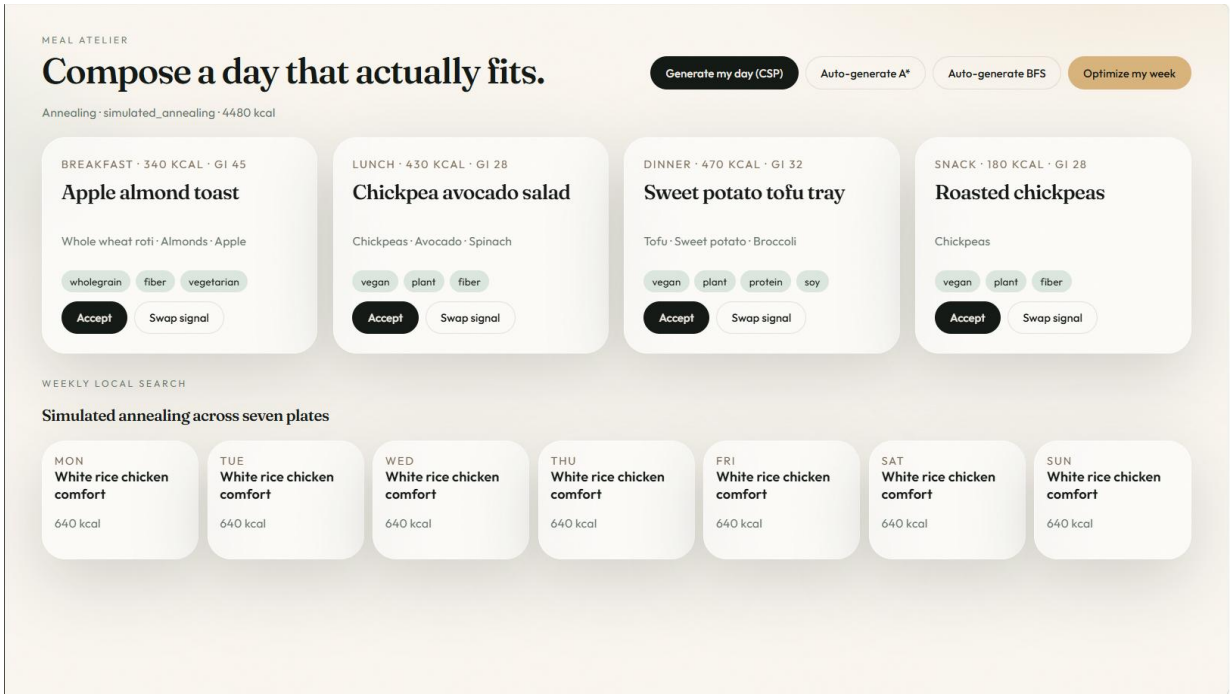
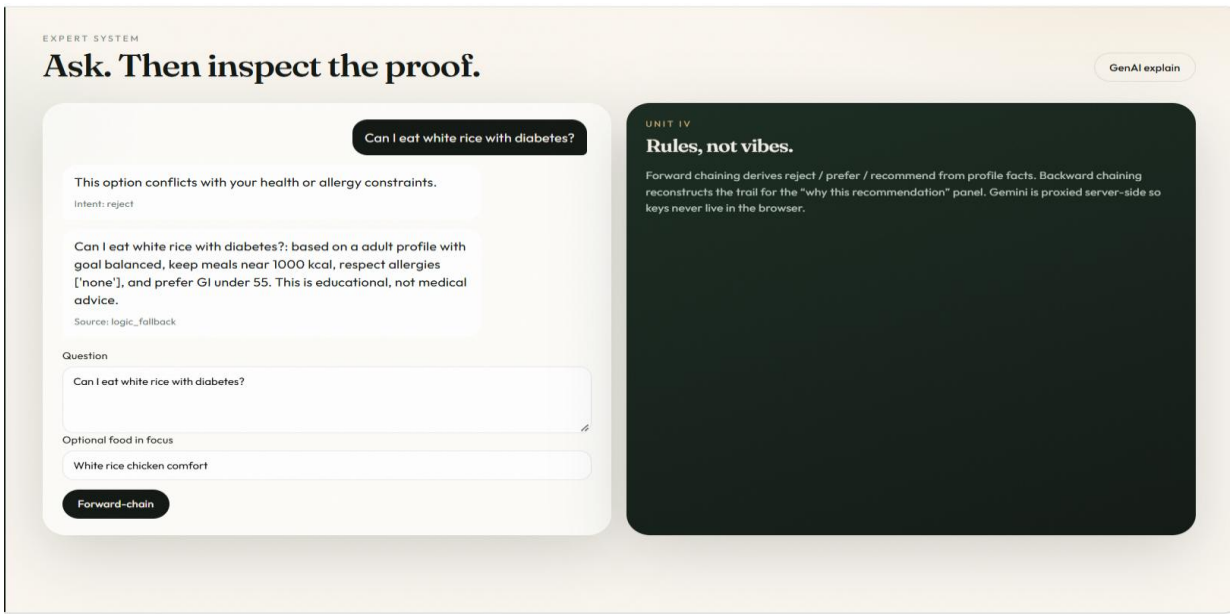


Fig 5.3 Meal Planner — CSP-Generated Day Plan



5.4 AI Chatbot — Recommendation with 'Why' Reasoning Trail

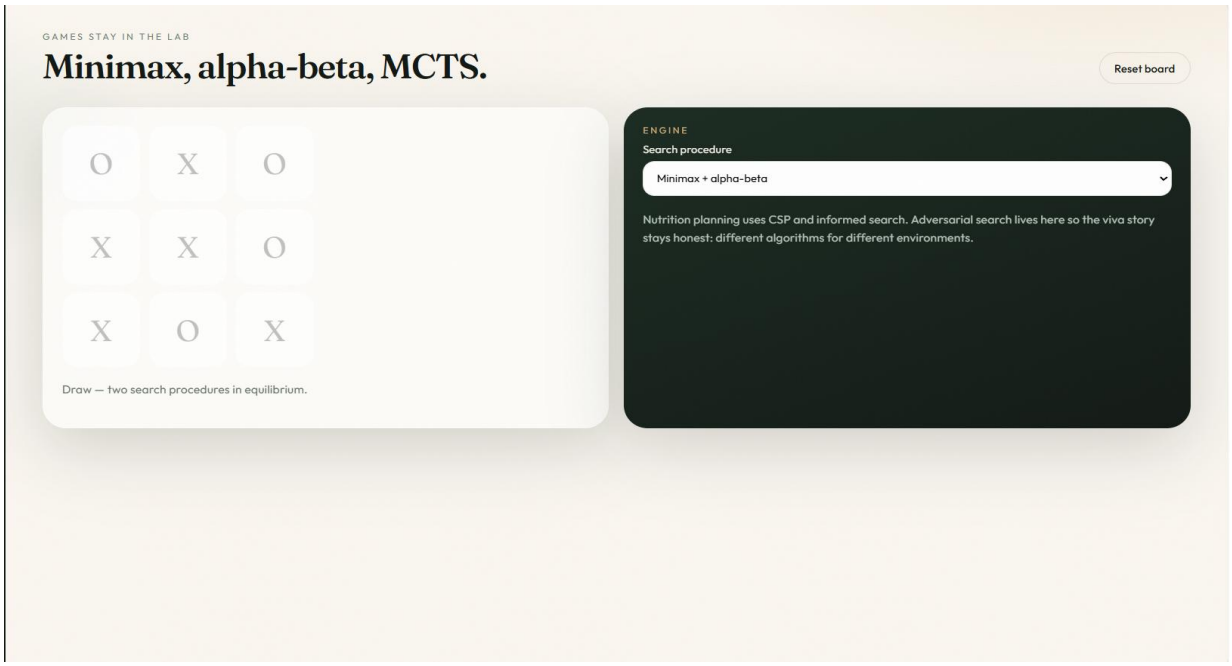


Fig 5.5 AI Lab — Minimax / Alpha-Beta / MCTS Game

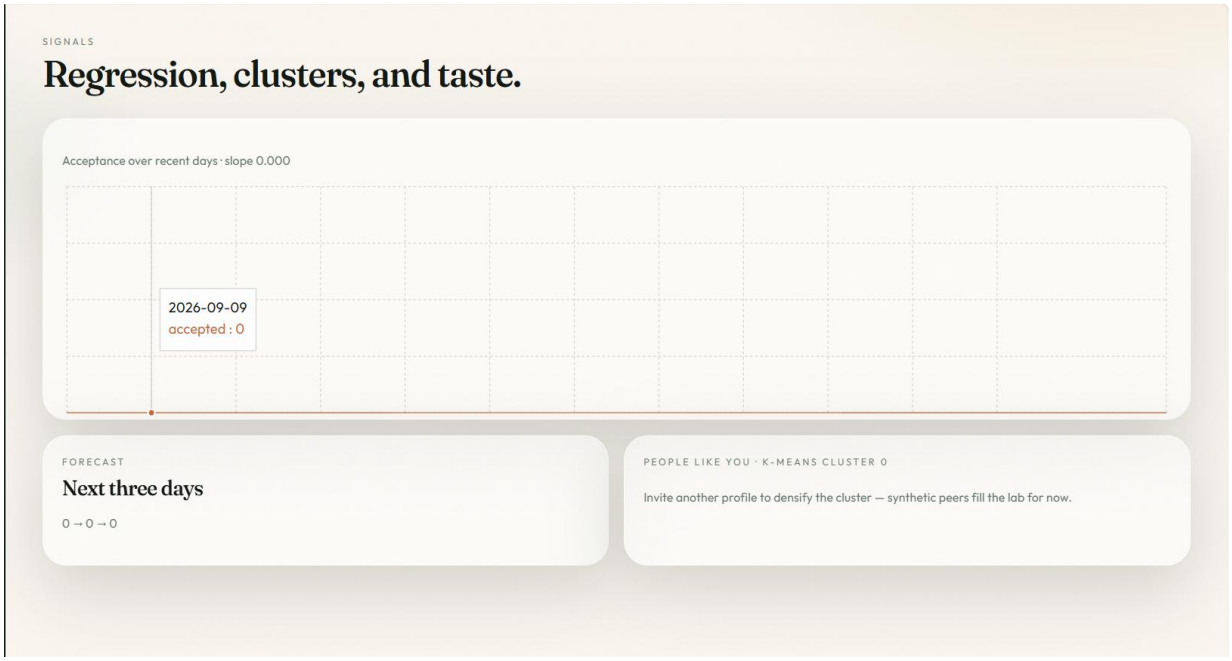


Fig 5.6 Analytics — Trend Prediction & Clustering

CHAPTER 6

CONCLUSION AND FUTURE ENHANCEMENT

Through the development of NutriGuide AI, users are able to receive meal and workout recommendations that are personalized, explainable, and provably consistent with their stated health constraints. The system demonstrates a working, integrated application of classical Artificial Intelligence techniques — intelligent agents, uninformed and informed search, local search optimization, constraint satisfaction with propagation, adversarial game-tree search, propositional logical inference, and machine learning — rather than treating them as isolated classroom exercises. By keeping the algorithms package independent of the web framework and database layer, each technique remains individually testable and directly traceable to the syllabus unit it implements.

In the future, the system can be enhanced with real computer-vision-based food recognition (replacing the current simulated confidence scores), wearable-device integration for real-time activity and heart-rate data, a full Q-learning-based recommendation policy in place of the current lightweight reinforcement-learning weight update, and community features that let users share and compare meal plans. Ultimately, this project aims to show that a genuinely reasoning-driven recommendation engine is both feasible and preferable to a purely static or purely generative approach for health-sensitive applications.

. **REFERENCES**

1. Russell, S. and Norvig, P., Artificial Intelligence: A Modern Approach, 4th Edition, Pearson.
2. https://fastapi.tiangolo.com/https://www.wikipedia.org/
3. https://motor.readthedocs.io/
4. https://react.dev/
5. https://ai.google.dev/